

**VIETNAM-KOREA UNIVERSITY OF INFORMATION AND
COMMUNICATION TECHNOLOGY**



COMPUTER AND ELECTRONICS ENGINEERING



TOPIC

**DESIGN AND IMPLEMENTATION OF A 32-BIT ALU
SYSTEM WITH RISC-V ARCHITECTURE ON A DE2
FPGA**

Student:	Truong Cong Hoang	Student ID: 22CE038
	Duong Hien Doan Nguyen	Student ID: 22CE055
	Ngo Hoang Phuc	Student ID: 22CE063
	Tran Minh Quân	Student ID: 22CE068

Course section: **Designing FPGA/ASICs with Verilog**

Instructor: **Dr. Duong Ngoc Phap**

Da Nang, May 2026

**VIETNAM-KOREA UNIVERSITY OF INFORMATION AND
COMMUNICATION TECHNOLOGY**



COMPUTER AND ELECTRONICS ENGINEERING



TOPIC

**DESIGN AND IMPLEMENTATION OF A 32-BIT ALU
SYSTEM WITH RISC-V ARCHITECTURE ON A DE2
FPGA**

Student:	Truong Cong Hoang	Student ID: 22CE038
	Duong Hien Doan Nguyen	Student ID: 22CE055
	Ngo Hoang Phuc	Student ID: 22CE063
	Tran Minh Quan	Student ID: 22CE068

Course section : **Designing FPGA/ASICs with Verilog**

Instructor: **Dr. Duong Ngoc Phap**

Da Nang, May 2026

COMMENTS

(By the instructor)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Da Nang, May 2026

Instructor

Dr. Duong Ngoc Phap

ACKNOWLEDGEMENTS

We would like to express our deepest gratitude to Professor Duong Ngoc Phap for his dedicated support, guidance, and direction throughout the research and implementation of the project "Design and Implementation of a 32-bit ALU System with RISC-V Architecture on DE2 FPGA".

The professor's insightful feedback on computer architecture, Verilog HDL design, and testing procedures was not only a valuable source of knowledge but also a tremendous motivation that helped us overcome the technical challenges during the project.

We also express our gratitude to the Board of Directors of the Vietnam-Korea University of Information and Communication Technology, and to the esteemed lecturers in the Faculty of Computer Engineering and Electronics for providing us with a solid foundation of knowledge in digital design and computer architecture throughout our studies.

Despite our best efforts, due to our limited knowledge and experience, this report inevitably contains shortcomings. We sincerely hope to receive comments and feedback from our professors so that we can improve this report.

Thank you!

Table of contents

COMMENTS	iii
ACKNOWLEDGEMENTS	iv
CATEGORY OF ABBREVIATIONS	vii
LIST OF TABLES AND FIGURES	viii
LIST OF FIGURES	x
INTRODUCTION	xi
1. Research context.....	10
2. Research Objectives.....	10
3. Scientific and practical significance.....	11
4. Research Methodology.....	12
CHAPTER 1: THEORETICAL FOUNDATIONS	13
1.1 General Introduction to ALU.....	13
1.2 RISC-V Architecture and RV32I Instruction Set.....	14
1.3 Instruction Set Analysis – ALU Control Code Table.....	15
1.4 Basic Structure of an ALU.....	16
1.4.1 Arithmetic Unit	16
1.4.2 Logic Unit	16
1.4.3 Extended Operations in ALU RISC-V	16
1.4.4 Comparator and Flags	17
1.5 Introduction to Cyclone II FPGA and DE2 Board.....	17
1.5.1 FPGA Cyclone II EP2C35F672C6	18
1.6 Comparison of 8-bit and 32-bit RISC-V ALUs.....	19
CHAPTER 2: SYSTEM ANALYSIS AND DESIGN	20
2.1 Overall System Design.....	20
2.1.1 Input Block – Top-Core Input Control	21
2.1.2 ALU Core	22
2.1.3 Control-Path	23
2.1.4 Display Block	23
2.2 Signal Layout.....	24
2.3 Hardware Model – Assigning DE2 Pins.....	26
2.4 Data-Path and Control-Path	29
2.4.1 Data-Path.....	29

2.4.2 Control-Path	30
2.5 ALU Input and Execution Control Mechanism.....	31
CHAPTER 3: VERILOG HDL DESIGN	34
3.1 Module Hierarchy Structure.....	34
3.2 ALU Core Module (ALU.v).....	34
3.3 Control Unit and Register Module (ControlUnit.v & RegFile.v).....	34
3.4 Datapath.v Module.....	35
3.5 TopCore.v Top-Level Communication Module.....	35
3.6 System testing testbench (tb_TopCore.v)	36
CHAPTER 4: RESULTS AND EVALUATION	38
4.1	Simulation
Results.....	38
4.2 Actual test results on the DE2 board.	42
4.3 Evaluation of the 32-bit RISC-V ALU.....	43
4.4 Suggestions for Improvement.....	44
4.5 Firmware-Based RISC-V Execution Verification.....	45
CONCLUSION AND FUTURE DEVELOPMENT	48
REFERENCES	49

CATEGORY OF ABBREVIATIONS

STT	Phrase	Abbreviate
1	Arithmetic Logic Unit	ALU
2	Field Programmable Gate Array	FPGA
3	Reduced Instruction Set Computer – Fifth Generation	RISC-V
4	RISC-V 32-bit Integer Instruction Set	RV32I
5	Hardware Description Language	HDL
6	Verilog Hardware Description Language	Verilog HDL
7	Register Transfer Level	RTL
8	Program Counter	PC
9	Instruction Memory	IMEM
10	Control Unit	CU
11	Register File	RegFile / RF
12	Instruction Set Architecture	ISA
13	Central Processing Unit	CPU
14	Read Only Memory	ROM
15	Random Access Memory	RAM
16	Multiplexer	MUX
17	Flip-Flop	FF
18	Light Emitting Diode	LED
19	Red Light Emitting Diode Array	LEDR
20	Green Light Emitting Diode Array	LEDG
23	Finite State Machine	FSM
24	Joint Test Action Group	JTAG
25	Seven-Segment Display	HEX
26	Universal Serial Bus	USB
27	System on Programmable Chip / SRAM Object File	SOF

28	Memory Initialization File	MIF
29	Slide Switch	SW

LIST OF TABLES AND FIGURES

Table 1.1 – ALU control code table and RISC-V RV32I command source	15
Table 2.1 – Input/Output (I/O) Signal Table of a 32-bit ALU System	25
Table 2.2 – Pin Mapping Table on DE2 Board	27
Table 2.3 – Input Control and ALU Execution Event Table	33
Table 4.1 – Simulation results of ALU and firmware-based test cases	42
Table 4.2 – Resource Evaluation Table for Cyclone II EP2C35F672C6 FPGA	44
Table 4.3 – Firmware instruction sequence and expected ALU results	45

LIST OF FIGURES

Figure 1.1 – RISC-V single-cycle CPU model and ALU location in the Data-Path	17
Figure 1.2 – Overall architecture of the Cyclone II EP2 FPGA	18
Figure 1.3 – Board DE2 Development Kit – Terasic Technologies	19
Figure 2.1 – Overall block diagram of the 32-bit ALU system on DE2	20
Figure 2.2 – Internal architecture of ALU Core with case-based operation selection	21
Figure 2.3 – Signal Interface Diagram	22
Figure 2.4 – Data-Path and Control-Path diagrams of the system	23
Figure 2.5 –Control Flow Diagram for Data Input and ALU Execution	24
Figure 4.1: Overview of the top-level RTL of the system synthesized from Quartus ii.	25
Figure 4.2: Input data latch control circuit and ALU central processing unit.	26
Figure 4.3 Close-up view of the combined decoder circuit structure controlling the 7-segment LED display system.	27
Figure 4.4 Automated report transcript page confirming ALU operations.	30
Figure 4.5 Waveform diagram illustrating the signal flow and the timing of system result closing.	32
Figure 4.6 Clock configuration summary	33
Figure 4.7: Report of the maximum operating frequency Fmax of the designed circuit .	34
Figure 4.8: Critical path analysis report for the Slack setup system .	36
Figure 4.9 – RTL Viewer showing KEY[2] synchronization, step_enable, and RiscV_ALU_Core connection in TopCore_Firmware.	37
Figure 4.10 – ModelSim waveform and transcript for firmware-based RISC-V ALU core simulation.	39
Figure 4.11 – Full RTL Viewer view of the synthesized firmware-based top-level design.	41
Figure 4.12 – ModelSim transcript confirming PASSED status for ADD, SUB, AND, OR, XOR, and SLT.	42

INTRODUCTION

1. Research context

- In the era of rapid digital technology development, computing systems increasingly demand flexible processing capabilities with high performance and low power consumption. The open-source RISC-V (Reduced Instruction Set Computer – Fifth Generation) instruction set architecture, developed at the University of California Berkeley since 2010, has quickly become a top choice in academic research and next-generation microprocessor development. Unlike closed commercial architectures, RISC-V does not require licensing and is designed on a modular principle, making it extremely convenient for research, teaching, and customized development.
- The Arithmetic Logic Unit (ALU) is the central component, considered the "heart" of any microprocessor architecture. In a single-cycle CPU model, the ALU handles all execution of operations from the RV32I instruction set – the 32-bit base integer instruction set of RISC-V. Designing, optimizing, and implementing the ALU according to the RV32I specification on an FPGA platform is a crucial step in translating computer architecture theory into actual hardware. Altera's DE2 FPGA platform with the Cyclone II EP2C35F672C6 chip provides an ideal environment for testing digital designs, allowing intuitive interaction with the processing core through switches, buttons, and displays.

2. Research Objectives

- This research aims to design, simulate, and implement a 32-bit ALU system based on the RISC-V RV32I architecture on a DE2 FPGA board. In the updated version, the project includes two verification modes: a manual ALU testing mode using switches and push buttons, and a firmware-based execution mode in which a small RISC-V

instruction sequence is loaded into Instruction Memory and executed through a simplified PC -> Instruction Memory -> Control Unit -> Register File -> ALU Datapath.

- Firstly, the ALU Core is designed to operate on the principle of combinational circuitry, capable of performing six basic arithmetic operations: ADD, SUB, AND, OR XOR, and SLT. The SLT operation is used for less-than comparisons using signed numbers. Besides the 32-bit calculation results, the ALU Core also generates status flags including Zero, Carry, Overflow, and Negative to support the testing and evaluation of system performance.

- Secondly, a Top Core top-level hardware interface module is designed to connect the ALU core to peripherals on the DE2 board. This module allows data loading from the SW [17:0] switch system, operand latch, and operation code via push buttons, while using signal synchronization and edge detection mechanisms to limit the effects of mechanical noise during operation. Due to the limited number of physical switches, the system uses a method of inputting the low 18 bits from the switch and expanding with additional 0 bits at the high end to form a 32-bit operand.

- Third, the results display block is implemented using a combinational decoding circuit, allowing the conversion of 32-bit results to 8-digit hexadecimal numbers and displaying them on 8 HEX0–HEX7 7-segment screens. Additionally, a portion of the result data and status signals are mapped to a single LED system to aid observation and debugging during hardware testing.

- Fourth, the system's functionality was tested through two complementary verification flows. The first flow uses Model Sim-Altera testbenches to verify individual ALU operations. The second flow uses firmware-based simulation, where machine instructions stored in firmware. Hex are read by Instruction Memory and executed step by step by the RISC-V ALU core. After simulation, the TopCore_Firmware design was synthesized in Quartus II and deployed to the DE2 FPGA board for practical testing.

3. Scientific and practical significance

- Scientifically, this project contributes to realizing fundamental concepts in the fields of Computer Engineering and Electronics, such as Data-path and Control-path design, decoding RISC-V arithmetic operations, and building combinational circuits using the Verilog HDL hardware description language. The separation between the TopCore hardware interface module and the ALU Core processor core enables a modular design, facilitating testing, maintenance, and future functional expansion. Furthermore, the ALU design in this project can be used as a foundation for further integration with other components such as Register File, Instruction Memory, and Control Unit to develop a complete Single-Cycle RISC-V processor model.
- In practical terms, the system provides a visual hardware testing model on the DE2 FPGA platform, allowing users to directly observe the data input process, operation execution, and result display via LEDs and a 7-segment display. During implementation, the project simultaneously addresses common issues in digital hardware design such as button signal synchronization, handling mechanical noise, and ensuring data stability before finalizing results through an execute_delay mechanism.

4. Research Methodology

- The project implementation process followed the digital hardware design process on an FPGA platform, including steps from requirements analysis, architectural design, hardware description to simulation and practical testing. In the initial phase, the project focused on studying the RISC-V RV32I instruction set specification, thereby determining the necessary operations, input/output requirements, and data bus width of the ALU block.
- Next, the system is designed using a top-down design approach, where the overall functionality is divided into independent modules such as the ALU core, top-level communication block, and result display block. This approach facilitates simulation, testing, and future system expansion.
- After completing the Verilog HDL source code, the system was verified using ModelSim-Altera. The firmware-based testbench loads firmware.hex into Instruction

Memory, steps through each RISC-V instruction, and compares the ALU result with the expected result from the original firmware. Finally, the design was synthesized using Quartus II, inspected in RTL Viewer, assigned to DE2 board pins, and uploaded to the FPGA for hardware testing.

CHAPTER 1: THEORETICAL FOUNDATIONS

1.1 General Introduction to ALU

- The Arithmetic Logic Unit (ALU) is a crucial functional block in the processor, responsible for performing arithmetic and logical operations on binary data. In modern computer architecture, the ALU is part of the Data-Path component and operates under the control of the Control Unit.
- In a single-cycle RISC-V processor model, the ALU receives two 32-bit input operands to perform operations based on control signals from the instruction decoder. The first operand (SrcA) is typically read from the register file, while the second operand (SrcB) can come from a register or an immediate value via a sign extender. The ALU's output is used to write back to a register or to service the memory access address. The ALU's operation control signals are usually generated from decoding the opcode, funct3, and funct7 fields in the RISC-V instruction set.

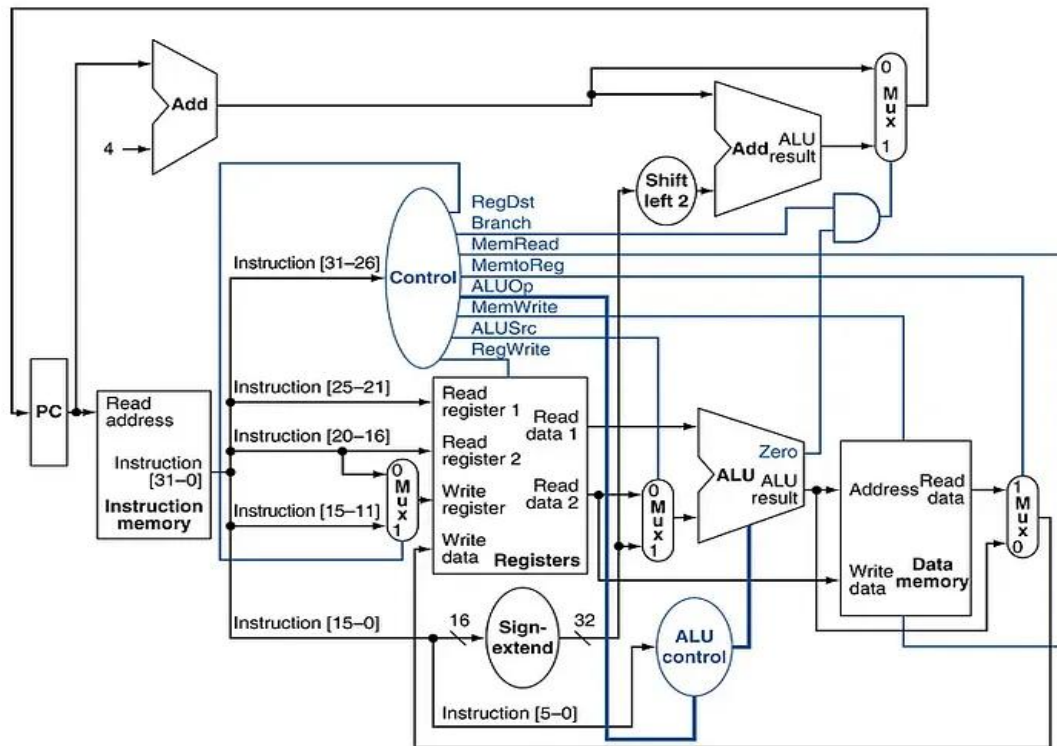


Figure 1.1 – RISC-V single-cycle CPU model and ALU location in the Data-Path 15

1.2 RISC-V Architecture and RV32I Instruction Set

- RISC-V is an open-source instruction set architecture developed based on the RISC philosophy. With features such as a simple, modular, and easily extensible structure. The RV32I base instruction set is a 32-bit version for basic integer operations.
- The RV32I defines several different instruction groups, including key formats such as R-type, I-type, S-type, B-type, U-type, and J-type. Each instruction format has its own encoding structure and is used for functions such as arithmetic calculations, memory access, or program flow control.
- In a single-cycle processor, the ALU performs most of the instruction set operations. R-type instructions such as ADD, SUB, AND, OR, XOR, and SLT perform operations directly between two source registers. I-type instructions such as ADDI, ANDI, ORI, XORI, and SLTI perform operations between a register and an immediate value. Additionally, the ALU is used to calculate memory addresses for LW/SW instructions

or check branch conditions for BEQ/BNE instructions through subtraction and the zero flag.

1.3 Instruction Set Analysis – ALU Control Code Table

- Based on the RISC-V RV32I architecture specification and the scope of this project, the ALU core is designed to support six basic operations: ADD, SUB, AND, OR, XOR, and SLT. These operations were selected to fulfill the core computational functions of an integer processor. Table 1.1 presents the 3-bit control code (`alu_sel`) corresponding to each and the related RV32I instructions.

Table 1.1 – ALU control code table and RISC-V RV32I command source

ALU Operation	ALU Control	Functional description	RV32I command source
ADD	000	$A + B$ (32-bit arithmetic addition)	ADD, ADDI, LW, SW
SUB	001	$A - B$ (subtraction using two's complement)	SUB, BEQ, BNE
AND	010	$A \text{ and } B$ (bitwise)	AND, ANDI
OR	011	$A \text{ OR } B$ (bitwise)	OR, ORI
XOR	100	$A \text{ XOR } B$ (bitwise)	XOR, XORI
SLT	101	1 if $A < B$ (with sign),	SLT, SLTI

ALU Operation	ALU Control	Functional description	RV32I command source
		otherwise equals 0	

- The Zero Flag is triggered when the ALU output is **0x00000000** . This flag is commonly used in branch instructions such as BEQ and BNE. Additionally, the Overflow flag is used to detect arithmetic overflow in addition or subtraction operations on signed data.

1.4 Basic Structure of an ALU

1.4.1 Arithmetic Unit

- The arithmetic block handles the ADD and SUB operations. In the design, both operations are performed using a 32-bit adder circuit: the addition is calculated directly as $A+B$.
- Meanwhile, subtraction is performed using the two's complement method:

$$A - B = A + (\sim B) + 1$$

- This design allows the use of a single 32-bit adder circuit for both operations, thereby reducing logic resource consumption on the FPGA. Within the scope of this project, the **Ripple-Carry Adder** meets the requirements for the 50 MHz frequency of the DE2 board.

1.4.2 Logic Unit

- The logic block performs bitwise operations including AND, OR, and XOR on each bit of the two 32-bit operands. This is a purely combinational circuit, built from basic logic gates and does not use state storage elements. Due to its simple structure, the propagation delay of the logic block is relatively small on the FPGA.

1.4.3 Extended Operations in ALU RISC-V

- In addition to the operations implemented within the scope of this project, the RV32I architecture also supports bit shift operations such as SLL (Shift Left

Logical), SRL (Shift Right Logical), and SRA (Shift Right Arithmetic). These operations use the 5 low bits of the B operand to determine the number of bits to shift.

- In this case, the SRL operation performs a logical right shift and inserts a 0 to the left, while the SRA operation performs an arithmetic right shift and preserves the sign bit of signed data. In Verilog HDL, the >>> operator combined with type casting \$signed() is often used to describe an arithmetic right shift.

- Currently, bit shift operations are not implemented in the ALU core and are considered a direction for expansion in future versions of the system.

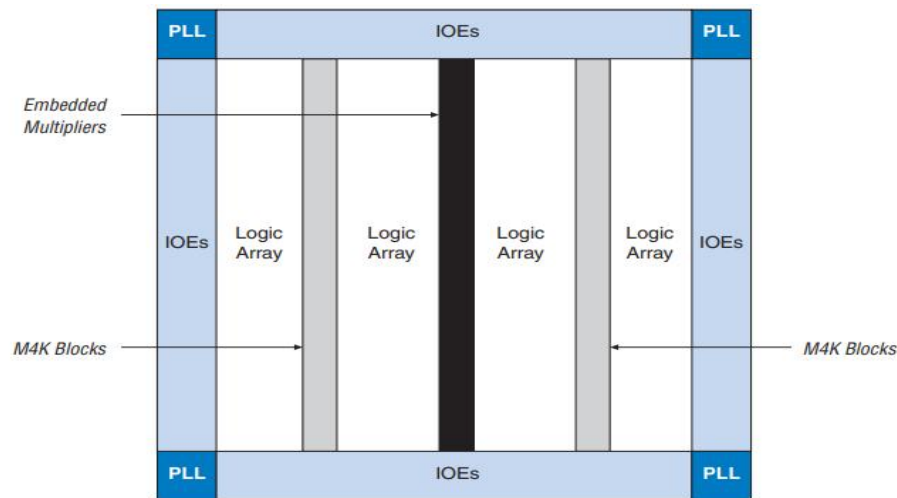
1.4.4 Comparator and Flags

- In this project, the Set Less Than (SLT) operator is used to compare two operands using signed numbers. If the value of A is less than B in two's complement representation, the result returned is 0x00000001 ; otherwise, the result returned is 0x00000000 .

- In addition to the calculation results, the ALU also generates status flags to support the system monitoring and control process:

- Zero Flag: Activated when the result is zero.
- Carry Flag: Reflects the carry bit of an extended addition or subtraction operation.
- Overflow Flag: Detects arithmetic overflow in signed data.
- Negative Flag: reflects the sign bit of the output result.

1.5 Introduction to Cyclone II FPGA and DE2 Board



1.5.1 FPGA Cyclone II EP2C35F672C6

Figure 1-2 – Overall architecture of the Cyclone II EP2 FPGA

- Altera's Cyclone II EP2C35F672C6 FPGA is a mid-range 90nm FPGA integrated into the DE2 board. This chip offers 33,216 Logic Elements (LEs), each LE comprising a 4-input lookup table (4-input LUT) and a flip-flop. Additionally, it features 483,840-bit Block RAM (M4K), 35 18x18-bit multipliers, and 4 Phase-Locked Loop (PLL) modules. The total available I/O is 475 pins, sufficient to connect all peripherals on the DE2 board.

It includes the following core components:

- **Logic Array:** This is where the LAB blocks and LE logic elements are located, playing a crucial role in implementing the entire combinational and sequential circuit structure of the RISC-V ALU.
- **M4K Block:** Arranged in vertical columns alternating between logic arrays. This is a high-speed memory resource used to store data or configure temporary registers.
- **Embedded Multiplier:** Arranged in a dedicated vertical column at the center of the chip core, it provides strong support for complex multiplication/division operations without consuming the resources of the Logic Array.

- IOEs: A system of interconnected peripheral interface blocks distributed across all four boards, which route the input/output signals of the ALU to the physical components on the DE2 board (Switch, LED, Key).
 - PLL: Located at the four corners of the chip, it is responsible for managing and distributing a stable clock network for the entire digital system.
- The Terasic Technologies DE2 board is a popular development and teaching platform in technical universities. The board integrates a full range of peripherals for teaching and research: 18 slide switches (SW), 4 mechanical push buttons (KEY, active-low), 18 red LEDs (LEDR), 9 green LEDs (LEDG), 8 7-segment displays (HEX0–HEX7), and expansion interfaces such as VGA, audio, SDRAM, SRAM, Flash, and Ethernet.

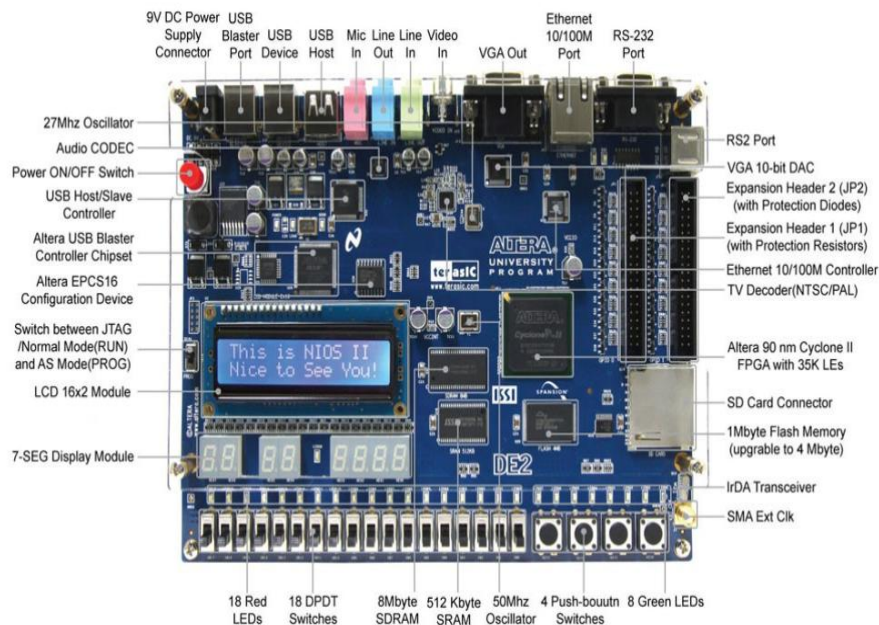


Figure 1.3 – Board DE2 Development Kit – Terasic Technologies

1.6 Comparison of 8-bit and 32-bit RISC-V ALUs

- Compared with a conventional 8-bit ALU, the 32-bit RISC-V ALU in this project has a wider data path and therefore requires full processing and display of the 32-bit result Result[31:0]. Within the scope of this project, the ALU supports six basic operations: ADD, SUB, AND, OR, XOR, and SLT. Bit-shift operations such as SLL, SRL, and SRA, as well as the unsigned comparison operation SLTU, have not been

implemented in the current version. Therefore, this design does not require a complete Barrel Shifter. Regarding the hardware interface, the DE2 board provides 18 slide switches SW[17:0], while each ALU operand has a width of 32 bits. Therefore, the manual test mode uses 18-bit input data from switches, zero-extends the upper bits to form a 32-bit operand, and latches the values into operand registers using KEY[0] and KEY[1]. The updated firmware mode further extends the project by allowing instruction words from firmware.hex to be executed through Program Counter, Instruction Memory, Control Unit, Register File, and ALU modules.

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN

2.1 Overall System Design

- The 32-bit RISC-V ALU system is designed with a clear hierarchical digital circuit architecture. The project now contains two top-level configurations. The first configuration, TopCore, supports direct manual ALU testing using SW[17:0] and KEY[3:0]. The second configuration, TopCore_Firmware, integrates a firmware-based RISC-V ALU execution path in which the Program Counter reads machine code from Instruction Memory and drives the ALU through a Control Unit and Register File.
- The system is divided into coordinated functional modules to support both hardware interaction and instruction-level verification. In the firmware-based configuration, the central flow is PC -> Instruction Memory -> Instruction Decode -> Control Unit -> Register File -> ALU -> Debug Display. This structure satisfies the requirement to demonstrate how the RISC-V core reads machine instructions and executes ALU operations.

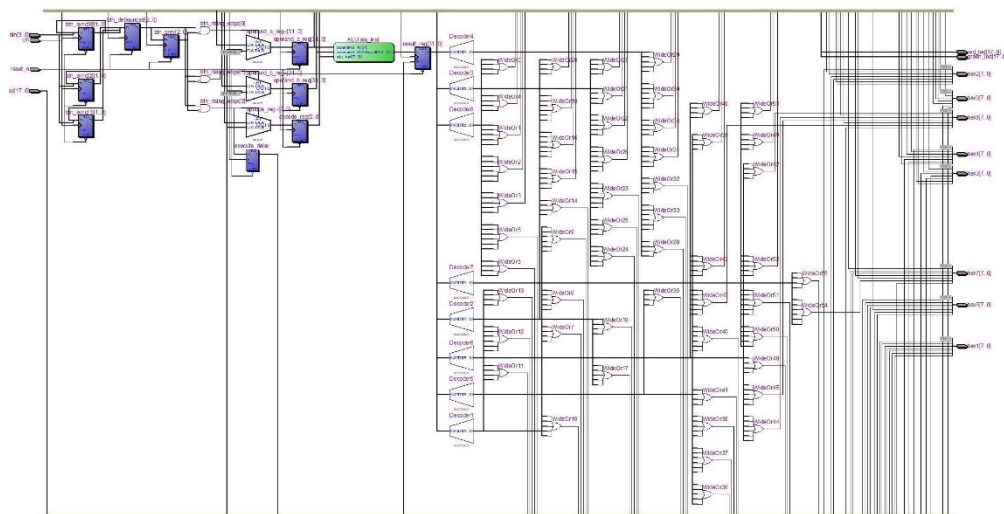


Figure 2.1 – Overall block diagram of the 32-bit ALU system on DE2

2.1.1 Input Block – Top-Core Input Control

- Because the DE2 board only provides 18 slide switches while the system uses 32-bit operands, the input control block is designed to receive data from the SW[17:0] and latch into the registers via push buttons.
- When the user presses KEY[0] , the system detects the rising edge of the signal and stores the data from SW[17:0] into operand register A (operand_a_reg). The missing bits in the high part are added with a value of 0 to form the 32-bit data. The same process is done with KEY[1] to load data into operand B (operand_b_reg).
- KEY[2] button is used to latch the 3-bit arithmetic code from SW[2:0] into the opcode_reg register . The button signals are passed through two Flip-Flop stages to synchronize with the system clock and reduce the effects of mechanical noise during operation.

2.1.2 ALU Core

- The ALU Core is a combinational circuit block that performs the arithmetic and logic operations of the system. Within the scope of this project, the ALU supports six operations: **ADD, SUB, AND, OR, XOR, and SLT** .
- The design utilizes a parallel computing method, where the results of each re generated simultaneously on independent logic branches. Then, the control signal alu_sel selects the corresponding result via a case structure to produce the final output result .
- Addition and subtraction operations are performed on 33-bit extended data to help determine the **Carry Flag** . In addition to the 32-bit result, the ALU also generates status flags including **Zero, Carry, Overflow, and Negative** to assist in system operation testing.

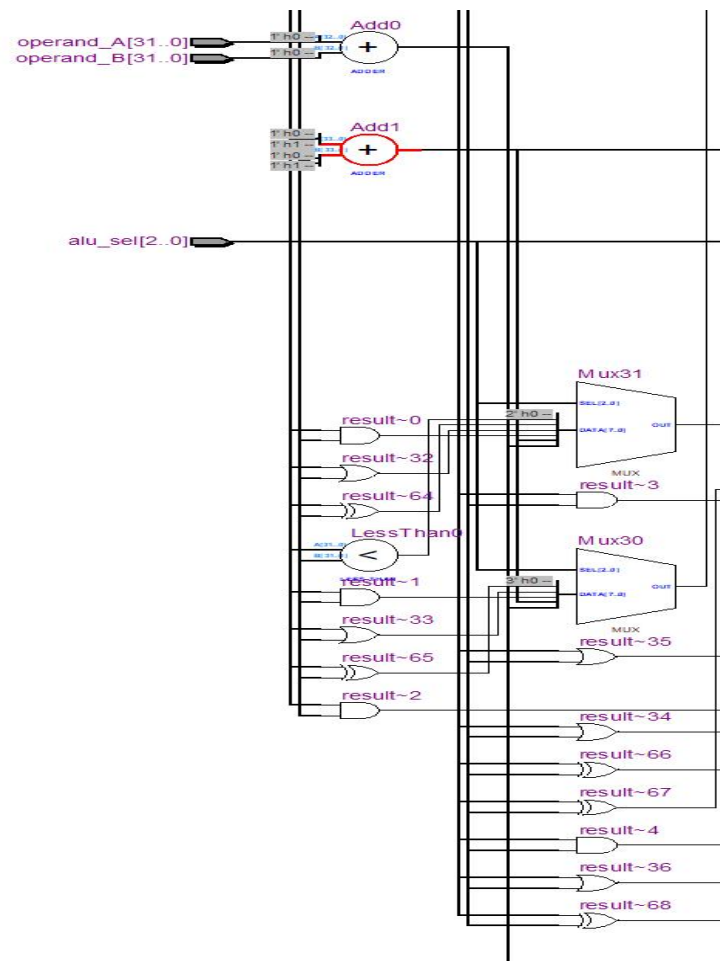


Figure 2.2 – Internal architecture of ALU Core with case-based operation selection

2.1.3 Control-Path

- The system's control block includes the command decoding module and the timing mechanism in TopCore. The 3-bit command code signal from opcode_reg is decoded into the control signal alu_sel to select the corresponding operation for the ALU.
- To ensure the result is recorded after the output of the combinatorial circuit has stabilized, the system uses a synchronous delay signal execute_delay . After the user presses KEY[2] , the result from the ALU will be latched into the result_reg register after one clock cycle. This design helps to limit the effect of signal propagation delay in the combinatorial circuit.

2.1.4 Display Block

- The result_reg value is displayed on the eight HEX0–HEX7 7-segment displays of the DE2 board as hexadecimal numbers.

- The system uses the hex_to_7seg combinational decoding function to convert each 4-bit group of data into the corresponding 7-segment LED control code. HEX0 displays the lowest 4 bits (Result[3:0]), and HEX7 displays the highest 4 bits (Result[31:28]).
- Additionally, the lower 18 bits of the result are output directly to the red LED array (LEDR) to aid in observing and inspecting the data during hardware testing.

2.2 Signal Layout

- Table 2.1 presents the main input/output signals of the 32-bit ALU system when implemented on the Altera DE2 board. The signals are organized into two main groups: data lines, control lines, and result display groups. In this design, the user inputs operands via switches SW[17:0] , controls the data loading and execution process using push buttons KEY[3:0] . The ALU results are displayed simultaneously on the red LED array LEDR [17:0] and the 7-segment LED displays HEX0 to HEX7 .

System Block Diagram: TopCore 32-bit RISC-V ALU System

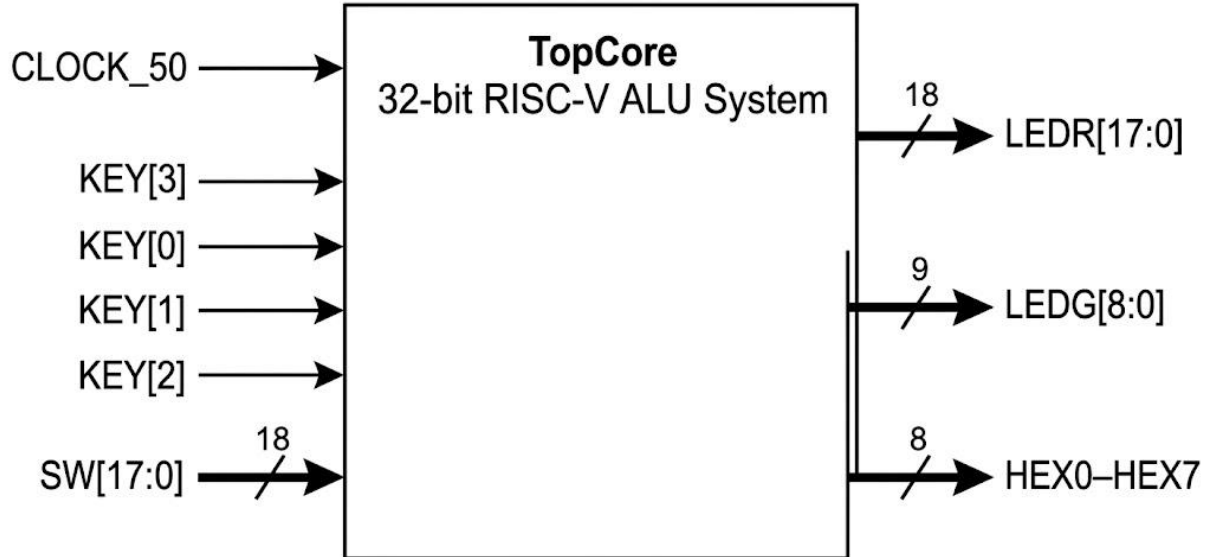


Figure 2.3 – Signal Interface Diagram

Table 2.1 – Input/Output (I/O) Signal Table of a 32-bit ALU System

Signal name	Bit	Direction	Functional description
CLOCK_50	1	In	The DE2 board's 50 MHz system clock is used to synchronize internal registers such as register A, register B, opcode register, and result register.
KEY[3]	1	In	The system reset button is active low. When KEY3 is pressed, registers A, B, opcode, result, and flag are reset to 0.
KEY[0]	1	In	The operand A is loaded. When KEY0 is pressed, the current data on SW[17:0] is latched into the operand_a_reg register .
KEY[1]	1	In	Operand B is loaded. When KEY1 is pressed, the current data on SW[17:0] is latched into the operand_b_reg register .
KEY[2]	1	In	The operation execution button. When KEY2 is pressed, the command code from SW[2:0] is latched into opcode_reg , then the ALU performs the nd the result is stored in result_reg .
SW[17:0]	18	In	Eighteen slide switches are used to input data for operand A, operand B, or ALU operation selection code via SW[2:0] .
LEDR[17:0]	18	Out	The red LED array displays the low 18 bits of the ALU result, corresponding to result_reg[17:0] .
LEDG[0]	1	Out	Display the Zero flag. LEDG0 lights up when the ALU result is 0.
LEDG[1]	1	Out	Display the Carry flag. LEDG1 lights up when the operation generates a Carry bit.
LEDG[2]	1	Out	Display the Overflow flag. LEDG2 lights up when a signed overflow occurs.
LEDG[3]	1	Out	Display the Negative flag. LEDG3 lights up when the result has a sign bit equal to 1.
LEDG[6:4]	3	Out	Display the current opcode latched into the opcode_reg register .
LEDG[8:7]	2	Out	Not used in the main function, assigned level 0.

Signal name	Bit	Direction	Functional description
HEX0	7	Out	Display the lowest nibble of the result, corresponding to <code>result_reg[3:0]</code> .
HEX1	7	Out	Display <code>result_reg[7:4]</code> .
HEX2	7	Out	Display <code>result_reg[11:8]</code> .
HEX3	7	Out	Display <code>result_reg[15:12]</code> .
HEX4	7	Out	Display <code>result_reg[19:16]</code> .
HEX5	7	Out	Display <code>result_reg[23:20]</code> .
HEX6	7	Out	Display <code>result_reg[27:24]</code> .
HEX7	7	Out	Display the highest nibble of the result, corresponding to <code>result_reg[31:28]</code> .

2.3 Hardware Model – Assigning DE2 Pins

- Hardware pin assignment plays a crucial role in implementing a 32-bit ALU system on the Altera DE2 board. Logic signals in the design are directly mapped to physical components on the board such as slide switches, push buttons, red LEDs, green LEDs, and 7-segment LED displays. This arrangement makes the demo process intuitive, allowing users to input operands, select operations, and observe the ALU's processing results in real time.

- In this design, the user enters data via SW[17:0] . The KEY[3:0] buttons are used to reset the system, load operand A, load operand B, and execute the operation. The ALU result is displayed simultaneously on the red LED array LEDR[17:0] and the 7-segment LED displays HEX0–HEX7 . In addition, the current status flags and opcode are displayed on the green LED array LEDG[8:0] .

Table 2.2 – Pin Mapping Table on DE2 Board

Signal/Pin DE2	Direction	DE2 board components	Describe
CLOCK_50	In	50 MHz oscillator	Main system clock signal
KEY[3]	In	Mechanical push button	Active-low system reset
KEY[0]	In	Mechanical push button	Load operand A into <code>operand_a_reg</code>
KEY[1]	In	Mechanical push button	Load operand B into <code>operand_b_reg</code>
KEY[2]	In	Mechanical push button	Execute ALU operation and store result
SW[17:0]	In	18 slide switches	Input data and opcode selection switches
LEDR[17:0]	Out	18 red LEDs	Display lower 18 bits of ALU result
LEDG[0]	Out	Green LED	Zero flag indicator
LEDG[1]	Out	Green LED	Carry flag indicator
LEDG[2]	Out	Green LED	Overflow flag indicator
LEDG[3]	Out	Green LED	Negative flag indicator
LEDG[6:4]	Out	3 LED	Display current opcode value
LEDG[8:7]	Out	2 green LEDs	Unused output (logic 0)
HEX0	Out	7-segment LED	Display <code>result_reg[3:0]</code>
HEX1	Out	7-segment LED	Display <code>result_reg[7:4]</code> .
HEX2	Out	7-segment LED	Display <code>result_reg[11:8]</code> .
HEX3	Out	7-segment LED	Display <code>result_reg[15:12]</code> .
HEX4	Out	7-segment LED	Display <code>result_reg[19:16]</code> .
HEX5	Out	7-segment LED	Display <code>result_reg[23:20]</code> .
HEX6	Out	7-segment LED	Display <code>result_reg[27:24]</code> .

Signal/Pin DE2	Direction	DE2 board components	Describe
HEX7	Out	7-segment LED	Display <code>result_reg[31:28]</code>

-Notes on the push-button control mechanism on the DE2

- The physical buttons on the DE2 board are **active-low** , meaning that when not pressed, the signal is at logic level 1; when pressed, the signal switches to logic level 0. Therefore, in the TopCore module, the KEY[2:0] signal is logic-inverted to create a high-level internal control signal when the user presses the button.

Specifically:

- `key_sync_0 <= ~KEY[2:0];`
- `key_sync_1 <= key_sync_0;`
- `key_prev <= key_sync_1;`

- After being synchronized through two Flip-Flop stages, the button signal is fed into the edge detection circuit:

- `assign key_pressed = key_sync_1 & ~key_prev;`

- This mechanism generates a control pulse with a clock cycle width upon detecting a button press. Therefore, each time the user presses `KEY[0]` , `KEY[1]` , or `KEY[2]` , the system performs only one corresponding action such as loading A, loading B, or executing the ALU.

- However, note that this mechanism primarily performs signal synchronization and edge detection, and is not a complete debounce circuit in the sense of using a debounce timer. In the context of the DE2 board demo, this approach is sufficient for stable operation if the user presses and releases the button decisively.

2.4 Data-Path and Control-Path

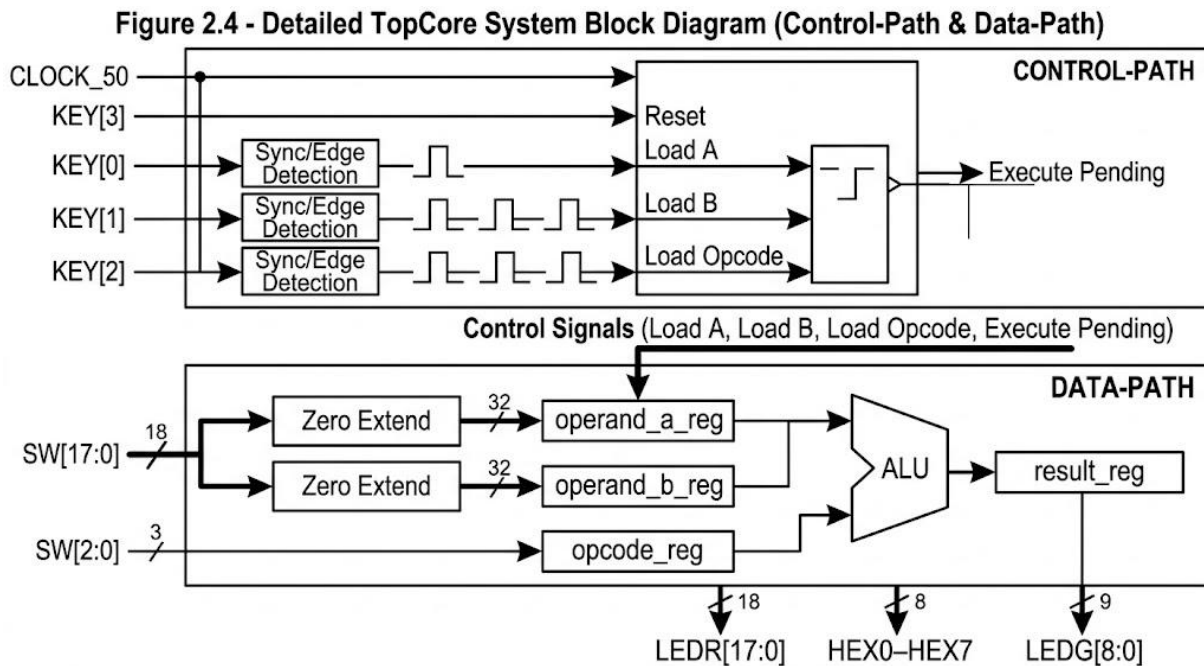


Figure 2.4 – Data-Path and Control-Path diagrams of the system

- The 32-bit ALU system is organized into two main parts: **Data-Path** and **Control-Path**. Data-Path handles the transmission, storage, and processing of data between registers and the ALU block. Control-Path is responsible for generating control signals based on user button presses on the DE2 board.

2.4.1 Data-Path

- The system's data path is designed for unidirectional data flow, suitable for direct testing on FPGA hardware. Input data is received from 18 slide switches **SW[17:0]**. Since the ALU handles 32-bit data, the 18-bit value from the switch is expanded by adding zeros to the high end to form 32-bit data before being loaded into the operand registers.

- When the user presses **KEY[0]**, the data from **SW[17:0]** is latched into the **operand_a_reg** register. Similarly, when **KEY[1]** is pressed, the data from **SW[17:0]** is latched into the **operand_b_reg** register. These two registers serve as temporary storage for the two input operands of the ALU.

- After the two operands have been saved, the user selects the operation using SW[2:0] . When KEY[2] is pressed, this instruction code is latched into the opcode_reg register . From there, operand_a_reg , operand_b_reg , and opcode_reg are passed to the ALU module to perform the corresponding operation such as ADD, SUB, AND, OR, XOR, or SLT.
- The combined result from the ALU is output to the result_alu signal . After a clock delay cycle via the execute_pending control signal , this result is latched into the result_reg register . Simultaneously, status flags such as zero_flag , carry_flag , overflow_flag , and negative_flag are also stored in their respective flag registers.
- At the output, result_reg[17:0] is displayed directly on the red LED array LEDR[17:0] . The entire 32-bit result in result_reg is divided into 8 groups of 4-bits and passed through the hex_to_7seg decoding function to display on 8 clusters of 7-segment LEDs HEX0 to HEX7 . Specifically, HEX0 displays the 4 lowest bits result_reg[3:0] , while HEX7 displays the 4 highest bits result_reg[31:28] .
- In addition, the current status flags and opcode are displayed on the green LED array LEDG[8:0] . In which, LEDG[0] represents the Zero flag, LEDG[1] represents the Carry flag, LEDG[2] represents the Overflow flag, LEDG[3] represents the Negative flag, and LEDG[6:4] represents the current opcode.

2.4.2 Control-Path

- The system's Control-Path does not use a complex finite state machine (FSM), but is built on the synchronization and edge detection mechanism of the KEY[2:0] buttons. On the DE2 board, the KEY buttons are **active-low** signals , meaning that when not pressed, the signal is at logic level 1; when pressed, the signal drops to logic level 0. Therefore, in the design, the KEY[2:0] signal is logic-inverted to create a high internal signal when the user presses the button.
- The button signals, after being inverted, are passed through two Flip-Flop stages: key_sync_0 and key_sync_1 . These two stages are responsible for synchronizing the button signals with the clock pulse CLOCK_50 , helping to minimize the impact of asynchronous signals from external hardware.

-Next, the `key_sync_1` signal is compared with the previous state stored in `key_prev` to generate the `key_pressed` edge detection signal . This signal is only one clock cycle wide and is used as a control pulse for data loading and operation execution.

- Specifically:

- `key_pressed[0]` → Load data into `operand_a_reg`
- `key_pressed[1]` → load data into `operand_b_reg`
- `key_pressed[2]` → Load opcode and enable ALU execution

- `Execute_pending` signal . When the user presses `KEY[2]` , the system first latches the opcode from `SW[2:0]` into `opcode_reg` . Since `opcode_reg` is a synchronous register, the new opcode value only takes effect after the clock edge. Therefore, the `execute_pending` signal is used to delay latching the ALU result by one clock cycle. As a result, `result_reg` only receives the result after `opcode_reg` has been updated and the ALU has calculated according to the new opcode.

- It should be noted that this mechanism performs well in synchronization **and edge detection** , but it is not a complete debounce circuit in the sense of using a multi-cycle debounce counter. In the demo on the DE2 board, decisive button presses and releases ensure system stability and prevent data duplication.

2.5 ALU Input and Execution Control Mechanism

- The system is designed with an event-driven control mechanism synchronized with the system clock (`clk`) at 50 MHz. All registers storing data and control signals are updated on the rising edge of the clock. Additionally, the system supports an active low `reset_n` signal to return the registers to their initial state.

- Instead of using a sequential finite state machine (FSM) to control the data input process, the design utilizes an event detection mechanism from push buttons on the DE2 board. Mechanical push button signals are passed through two Flip-Flop stages to synchronize with the system clock domain and reduce the impact of mechanical noise.

Then, a combinational logic circuit detects the rising edge to generate enable pulses with a width of one clock cycle.

-KEY0 is detected, the system latches the data from SW[17:0] into the operand_a_reg register. Similarly, the rising edge of KEY1 is used to load data into operand_b_reg. For the operation code, the rising edge of KEY2 latches the value SW[2:0] into the opcode_reg register and simultaneously triggers the execute_delay signal.

execute_delay signal is used to delay updating the result register by one clock cycle after the operation code has been established. This mechanism helps ensure that the output of the ALU combinational circuit has stabilized before data is written to result_reg and displayed on an LED or 7-segment display.

- Event-driven design allows users to change each operand or operation code independently without adhering to a fixed state sequence. This is suitable for testing directly on the DE2 FPGA hardware.

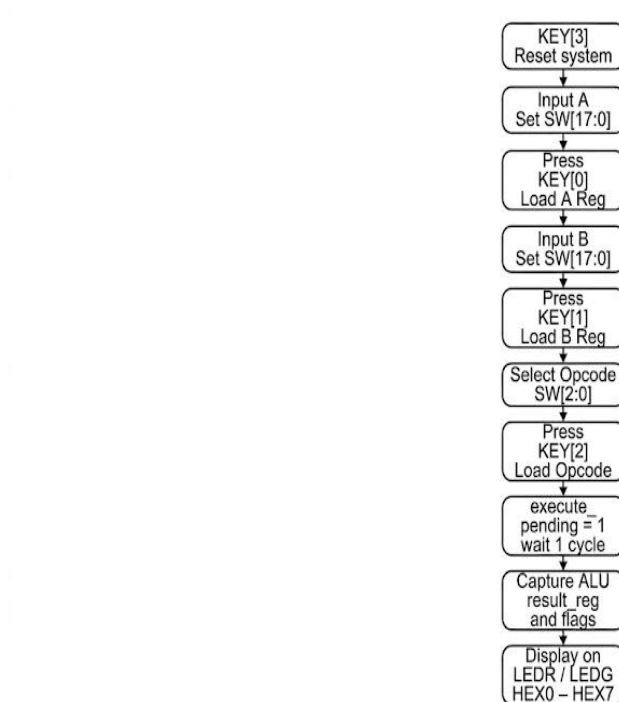


Figure 2.5 - Control Flow Diagram for Data Input and ALU Execution

Operation event	Logical conditions	Hardware register behavior/update
-----------------	--------------------	-----------------------------------

Operation event	Logical conditions	Hardware register behavior/update
Initialize the system.	<code>reset_n == 0</code>	Clear <code>operand_a_reg</code> , <code>operand_b_reg</code> , <code>opcode_reg</code> , <code>result_reg</code> , and flags
Load operand A	<code>btn_rising_edge [0] == 1</code>	Latch SW[17:0] into <code>operand_a_reg</code> and zero-extend to 32 bits
Load operand	<code>btn_rising_edge [1] == 1</code>	Latch SW[17:0] into <code>operand_b_reg</code> and zero-extend to 32 bits
Load opcode	<code>btn_rising_edge [2] == 1</code>	Latch SW[2:0] into <code>opcode_reg</code> and activate <code>execute_pending</code>
Capture result	<code>execute_delay == 1</code>	Latch <code>result_alu</code> and status flags into output registers

Table 2.3 – Input Control and ALU Execution Event Table

- Event-driven control design allows for more flexible data input and operation execution compared to traditional sequential FSM mechanisms. The `btn_rising_edge` signals act as independent write-enabled pulses, allowing users to change operands or operation codes without repeating the entire input process.
- In addition, the `execute_delay` mechanism helps synchronize the update time of the results after the ALU output signal has stabilized, thereby limiting the impact of propagation delay in the combinatorial circuit when implemented on actual FPGA hardware.

CHAPTER 3: VERILOG HDL DESIGN

3.1 Module Hierarchy Structure

- The design was built using the **Verilog HDL hardware description language** in a modular hierarchy to facilitate simulation, testing, and system expansion. The main modules in the system include:

- TopCore.v : a top-level module that communicates directly with the DE2 FPGA board.
- ALU.v : The ALU core performs arithmetic and logical operations.
- ControlUnit.v : a block for decoding control signals for mathematical operations.
- RegFile.v : a 32-bit register set for storing data.
- Datapath.v : a module that connects data streams between functional blocks.
- tb_TopCore.v : a testbench used for simulating and testing system functionality.

- This hierarchical structure helps separate the data processing part from the hardware communication part, thereby supporting the testing of each module independently before integrating it into the entire system.

3.2 ALU Core Module (ALU.v)

- This module performs 32-bit arithmetic and logic calculations through a pure combinational circuit.

- Supported operations: The module defines 6 operations through the localparam constant structure: ALU_ADD (3'b000), ALU_SUB (3'b001), ALU_AND (3'b010), ALU_OR (3'b011), ALU_XOR (3'b100), and ALU_SLT (3'b101).
- Flag extraction technique: Add_result and subtraction operations are performed on a 33-bit extended architecture ({1'b0, operand_A}) to accurately extract the carry_flag. The arithmetic overflow flag overflow_flag is determined by a logical expression comparing the signs of the input operands and the sign bit of the result.

3.3 Control Unit and Register Module (ControlUnit.v & RegFile.v)

- The ControlUnit module is designed as a purely asynchronous combinational circuit using an always @* block structure in conjunction with the case branch instruction. This block is responsible for receiving 3-bit opcode instructions from the system and

instantaneously translating them into a 3-bit `alu_sel` control signal to accurately route tasks to the arithmetic core within the ALU. Simultaneously, the control block is also responsible for generating the `reg_write_en` enable signal to a high logic level for all valid operations, preparing the conditions for data update activation.

- Alongside the control unit, the RegFile register set module is implemented as a static memory array consisting of 32 registers with 32-bit data size. The operation of the register set strictly adheres to modern digital circuit design principles, including two independent synchronous and asynchronous mechanisms. For data writing, the system applies synchronous sequential logic activated on the rising edge of the system clock when a `write_en` enable signal is received, and integrates a system reset circuit via the `reset_n` pin, which asynchronously operates at a low active level to erase the data of all 32 registers to zero. Conversely, the data reading process for the operands `operand_a` and `operand_b` is performed entirely via an asynchronous combinational circuit mechanism through independent read gates, allowing data from the registers to be immediately output to the ALU computing core without consuming any clock cycles.

3.4 Datapath.v Module

- Datapath.v module acts as a data link connecting the functional blocks within the system, including:

- RegFile
- ControlUnit
- ALU

- The two operands read from the register set are fed to the ALU's input to perform the operation according to the control signal `alu_sel`. After the calculation is complete, the result can be fed back to the storage system or sent to the display unit.

- Datapath design helps to clearly model the data flow in digital systems and provides a foundation for scaling up to a complete processor architecture in subsequent studies.

3.5 TopCore.v Top-Level Communication Module

- TopCore.v module is the top-level module of the entire system, responsible for connecting the logic processing blocks to peripheral devices on the DE2 FPGA board such as switches, push buttons, LEDs, and 7-segment displays.

- One of the module's main tasks is to process signals from mechanical push buttons. Input signals are passed through two Flip-Flop stages to synchronize with the system clock and reduce the impact of mechanical noise. Then, a rising edge detection circuit generates trigger pulses with a clock cycle width to control the data latching process. The SW[17:0] switches, data will be loaded into the operand registers via the push buttons:

- KEY0 : Load operand A
- KEY1 : Load operand
- KEY2 : Load the mathematical operation code

- Because the system uses a combinational circuit for the ALU core, the TopCore module incorporates an execute_delay signal to delay updating the result by one clock cycle after the operation code is established. This mechanism helps ensure that the ALU output data is stable before being written to result_reg .

- The 32-bit result from the ALU is displayed via:

- red LED array (LEDR)
- 8 seven-segment displays (HEX0–HEX7)

- Each 4-bit block of data will be converted to a 7-segment LED control code via the hex_to_7seg combination decoding function .

3.6 System testing testbench (tb_TopCore.v)

- tb_TopCore.v module is used to simulate the operation of the entire system in the ModelSim-Altera software. The testbench generates clock pulses, triggers the reset signal, and simulates button presses corresponding to operand input and operation execution.

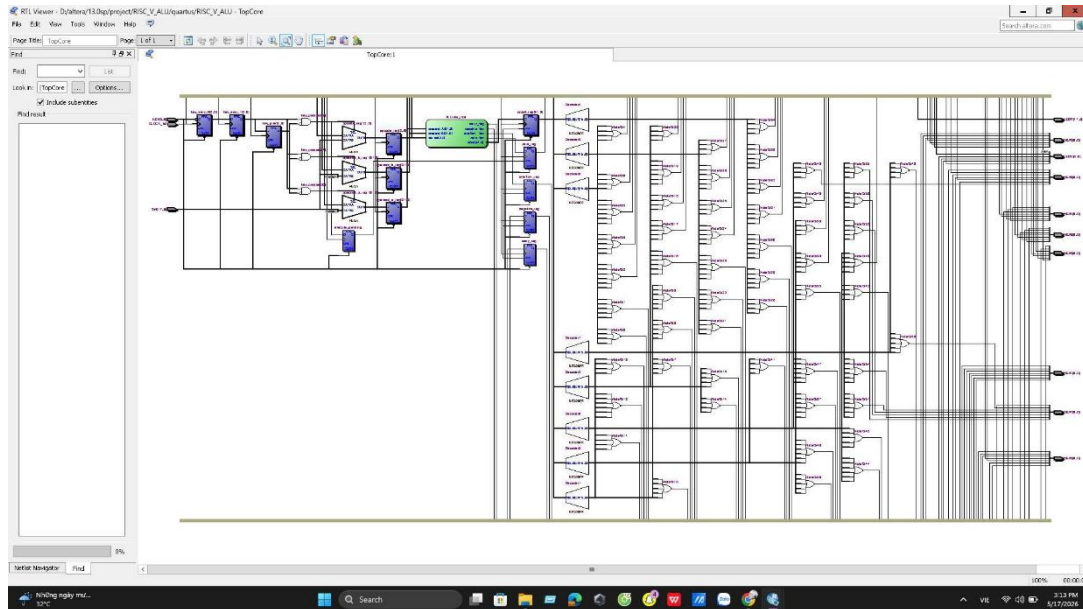
The test cases are designed to check:

- addition (ADD)

- Subtraction (SUB)
- logical operations (AND , OR , XOR)
- comparison (SLT)
- operation of status flags

The simulation results are observed through waveforms, which allows for the evaluation of the design's correctness before synthesis and deployment on a real FPGA.

CHAPTER 4: RESULTS AND EVALUATION



4.1 Simulation Results

Figure 4.1 Overview of the top-level RTL of the system synthesized from Quartus ii.

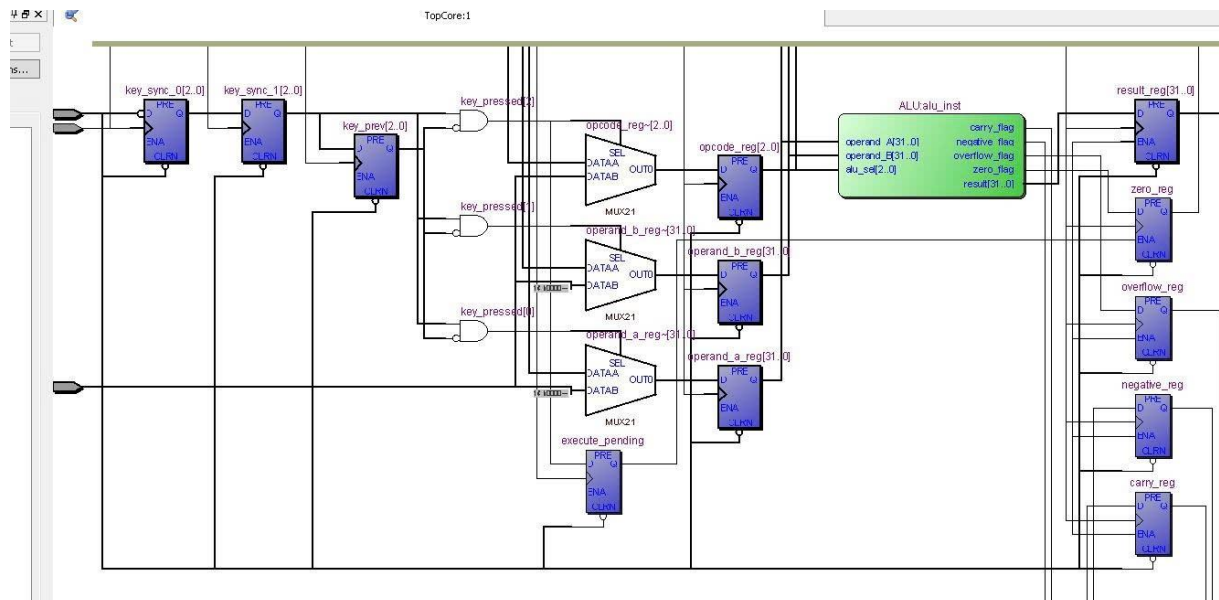


Figure 4.2: Input data latch control circuit and ALU central processing unit.

=> Key sync registers (key_sync, key_prev), operand registers (operand_a_reg, operand_b_reg), and the ALU block.

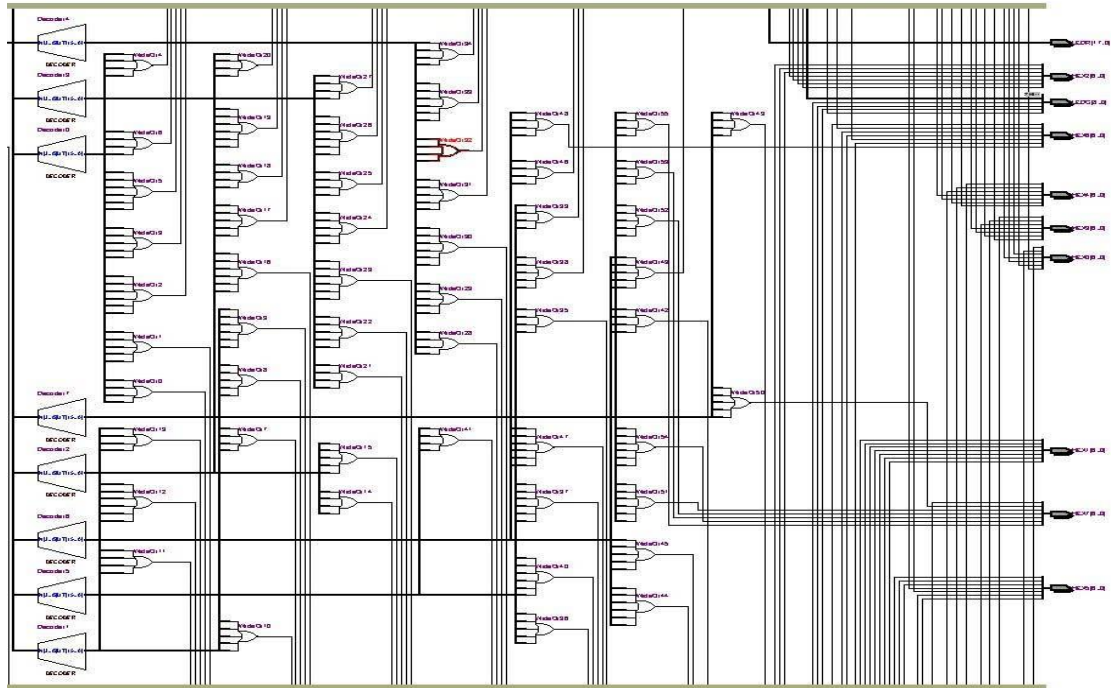


Figure 4.3 Close-up view of the combined decoder circuit structure controlling the 7-segment LED display system.

```
# tb_TopCore
ModelSim> vsim work.tb_TopCore
# vsim work.tb_TopCore
# Loading work.tb_TopCore
# Loading work.TopCore
# Loading work.ALU
VSIM6> add wave -r *
VSIM7> run -all
# =====
# Start TopCore Simulation - DE2 Version
# =====
#
# TEST 1: ADD 15 + 10
# [PASSED] ADD 15 + 10 | Result = 0x00019
#
# TEST 2: SUB 20 - 5
# [PASSED] SUB 20 - 5 | Result = 0x0000f
#
# TEST 3: AND 12 & 10
# [PASSED] AND 12 & 10 | Result = 0x00008
#
# TEST 4: OR 12 | 10
# [PASSED] OR 12 | 10 | Result = 0x0000e
#
# TEST 5: XOR 12 ^ 10
# [PASSED] XOR 12 ^ 10 | Result = 0x00006
#
# TEST 6: SLT 5 < 10
# [PASSED] SLT 5 < 10 | Result = 0x00001
#
# TEST 7: ZERO FLAG 10 - 10
# [PASSED] ZERO FLAG 10 - 10 | Result = 0x00000
# [PASSED] Zero flag LEDG[0] = 1
#
# =====
# ALL TESTS COMPLETED
# =====
# ** Note: $finish : tb_TopCore.v(266)
# Time: 10400 ns Iteration: 0 Instance: /tb_TopCore
# 1
# Break in Module tb_TopCore at tb_TopCore.v line 266
VSIM8>
```

Figure 4.4 Automated report transcript page confirming ALU operations.

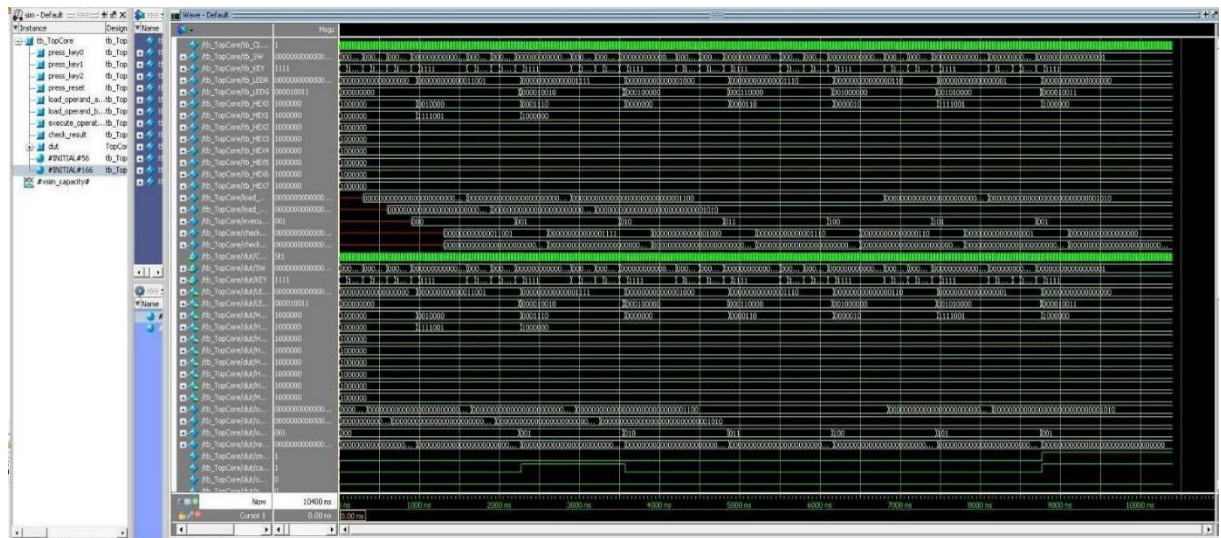


Figure 4.5 Waveform diagram illustrating the signal flow and the timing of system result closing.

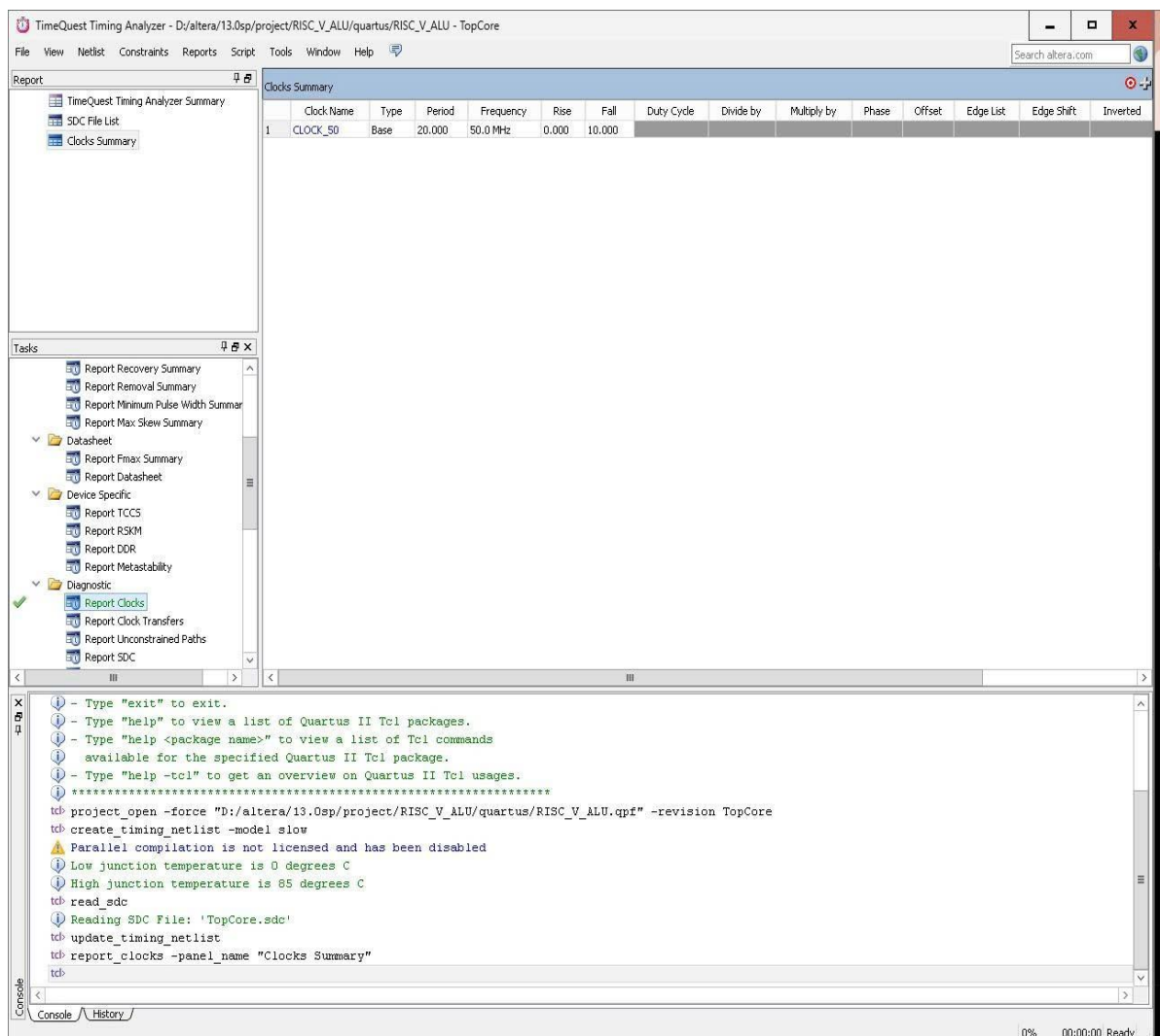


Figure 4.6 Clock configuration summary

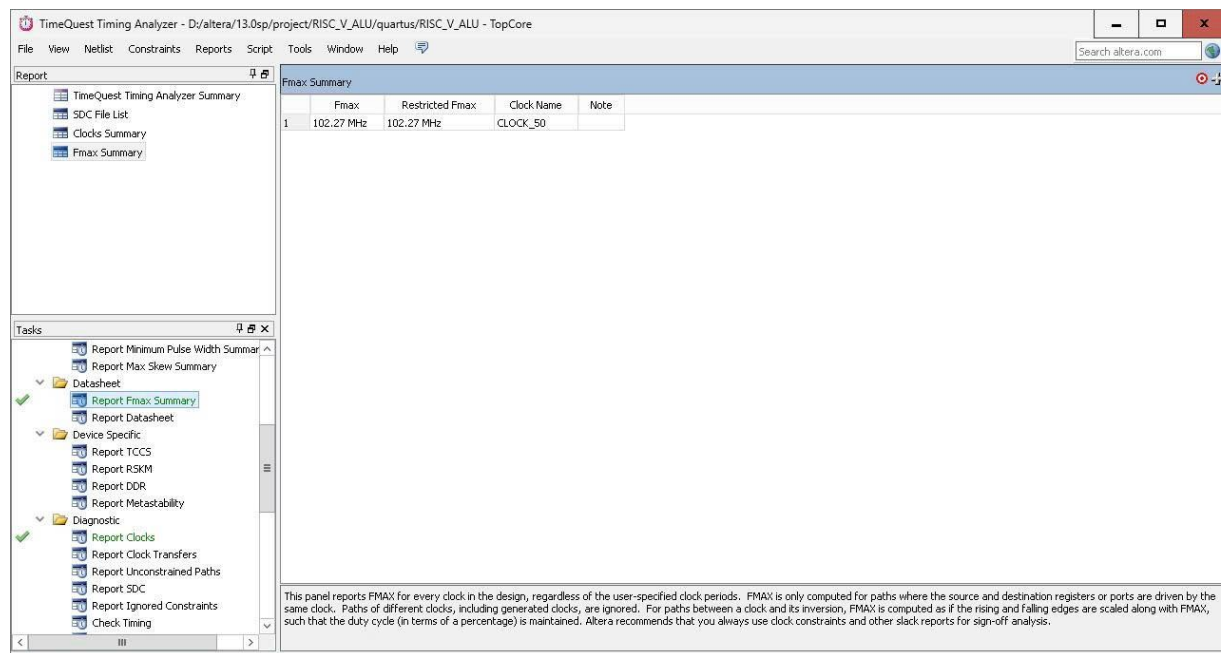


Figure 4.7: Report of the maximum operating frequency Fmax of the designed circuit .

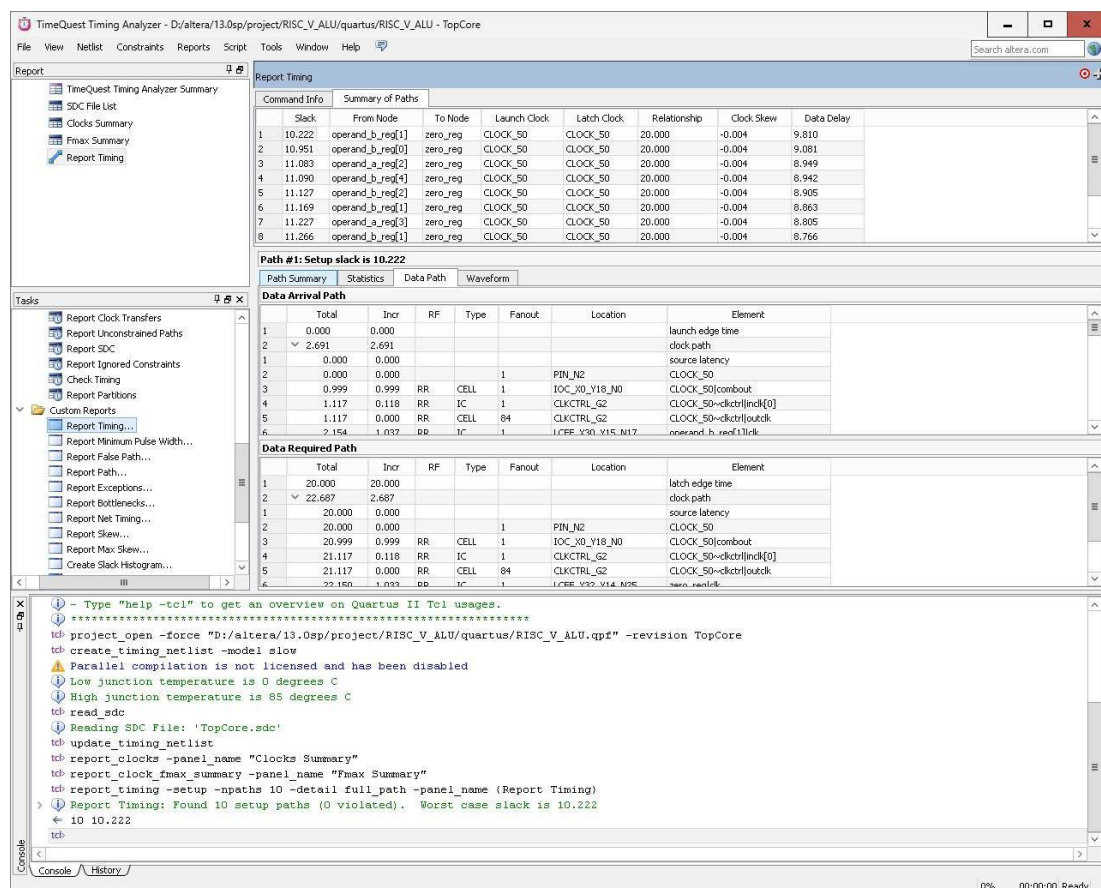


Figure 4.8: Critical path analysis report for the Slack setup system .

=> After synthesis, the system achieved a maximum operating frequency (Fmax) of up to 102.27 MHz, far exceeding the 50 MHz clock speed requirement of the DE2 board, demonstrating that the ALU synthesis circuit is optimally designed and suffers from minimal propagation delay.

Table 4.1 – Simulation results of ALU and firmware-based test cases

TC	Mathematical operations	A (Hex)	B (Hex)	Result (Hex)	Zero	Opcode	Note
01	ADD	0x0000 000F (15)	0x000 0000A (10)	0x00000019 (25)	0	000	Normal addition
02	SUB	0x0000 0014 (20)	0x000 00005 (5)	0x0000000F (15)	0	001	Normal subtraction, A > B.
03	AND	0x0000 000C (12)	0x000 0000A (10)	0x00000008 (8)	0	010	Logic AND (1100 & 1010 = 1000).
04	OR	0x0000 000C (12)	0x000 0000A (10)	0x0000000E (14)	0	011	Logical OR (1100 1010 = 1110).
05	XOR	0x0000 000C (12)	0x000 0000A (10)	0x00000006 (6)	0	100	XOR logic (1100 ^ 1010 = 0110).
06	SLT	0x0000 0005 (5)	0x000 0000A (10)	0x00000001 (1)	0	101	Comparison: 5 < 10 is true, output 1.
07	SUB	0x0000 000A (10)	0x000 0000A (10)	0x00000000 (0)	1	001	Subtraction of two equal operands, the Zero flag is set.

- Simulation results: All 7/7 test cases were completed with a PASSED status. Key points to note:

- TC06: Confirms the SLT-signed comparator circuit is functioning correctly. When operand A is less than B, the entire 32-bit result returns 0x00000001 .
- TC07: Confirm that the Zero status flag is synchronously active and immediately responds to logic level 1 when the result of the subtraction operation is 0 .
- All bitwise logic operations (AND, OR, XOR) are processed precisely on each bit independently.

4.2 Actual test results on the DE2 board

After successful functional simulation, the design is synthesized and uploaded to the Altera DE2 board via Quartus II software. The complete compilation process

includes: Analysis & Synthesis -> Fitter (Place & Route) -> Assembler (creates a .sof file) -> Programmer (loads via JTAG USB cable). After successful loading, the system immediately starts operating with a CLOCK_50 signal from the crystal oscillator on the board.

The procedure for testing an operation on DE2:

- Initialization: Press KEY[3] (reset_n) to clear all system registers to 0.
- Load operand A: Flip switches SW[17:0] to create a value for the number A. Press KEY[0] to latch. The system automatically expands by 14 bits of 0 in the high part to form a 32-bit number stored in operand_a_reg .
- Load operand B: Flip switches SW[17:0] to create a value for number B. Press KEY[1] to latch into operand_b_reg .
- Activate calculation: Flip the 3 switches SW[2:0] according to the desired operation code (Example: 000 enables Addition). Press KEY[2] to lock the command code. After a safe delay of 1 clock cycle, the result will be immediately locked in result_reg.
- Observation: Read the 32-bit Hexadecimal result directly on 8 seven-segment displays (HEX7 $\rightarrow$ HEX0). Observe the detailed bit state on the LEDR (low 18 bits) and LEDG (high 18 bits) arrays. You can directly change SW[2:0] and press KEY[2] again to see the results of other operations on the same operands without reloading from scratch.

4.3 Evaluation of the 32-bit RISC-V ALU

- The 32-bit RISC-V ALU design has been completed and has exceeded all the set objectives:

- Executed the six implemented ALU operations (ADD, SUB, AND, OR, XOR, and SLT) with correct results in both direct ALU testbench simulation and firmware-based RISC-V core simulation.
- The manual TopCore configuration uses synchronized button edge detection and a one-cycle execute delay rather than a complex multi-state FSM. The firmware

configuration uses KEY[2] as a step pulse to advance the Program Counter one instruction at a time.

- Displays full 32-bit results on eight 7-segment displays. In TopCore_Firmware, SW[1:0] selects display modes for ALU result, Program Counter, machine instruction, and debug information.
- Resource usage remains low on the Cyclone II EP2C35 FPGA even after adding Program Counter, Instruction Memory, Control Unit, Register File, and firmware debug display logic.

Table 4.2 – Resource Evaluation Table for Cyclone II EP2C35F672C6 FPGA

Resources	Estimated usage	Total available (EP2C35)	Utilization rate (%)
Logic Elements (LE)	~250 LE	33,216 LE	< 1%
Flip-Flops (Register)	~110 FF	33,216 FF	< 0.5%
Combinational Logic	~44 LE	-	-
Memory Bits	0 bit	483,840 bits	0%
DSP Multiplier 18×18	0	35 blocks	0%
Maximum frequency (Fmax)	102.27 MHz	-	Exceeding 50 MHz

4.4 Suggestions for Improvement

- Based on the experimental results obtained, the 32-bit ALU testing system on the DE2 FPGA opens up many potential development directions. The most important improvement is to further integrate the ALU core with the existing register file and Control Unit. By adding instruction memory, data memory, and program counter registers, the system will be completed into a single-cycle RISC-V microprocessor capable of directly executing standard RV32I machine code.

- To enhance computing performance, the core architecture can be extended to the RV32M instruction set by integrating dedicated multiplier modules, maximizing the resources of the Cyclone II chip's embedded multipliers. Simultaneously, the circuit

structure can be optimized for timing by replacing the current Ripple-Carry Adder with a Carry-Lookahead Adder. This will significantly shorten the critical path, allowing future processors to operate stably at higher clock frequencies.

- Finally, the interactive interface and hardware testing process also need upgrading. The system could integrate a 4x4 matrix keyboard to allow users to directly input hexadecimal numbers instead of manually flipping switches. Furthermore, adding a UART serial communication module would enable the board to transmit computational data directly to the computer, creating a favorable foundation for automated hardware testing.

4.5 Firmware-Based RISC-V Execution Verification

- To satisfy the final project requirement regarding firmware execution, the original manual ALU system was extended with a firmware-based verification path. The new top-level module TopCore_Firmware connects the DE2 board clock, push buttons, switches, LEDs, and HEX displays to a simplified RISC-V ALU core. In this mode, operands are no longer entered manually through switches. Instead, constants and ALU operations are encoded directly in firmware.hex and executed instruction by instruction.

- The firmware execution flow is organized as follows: Program Counter generates the instruction address; Instruction Memory reads a 32-bit machine instruction from firmware.hex; the instruction is decoded into opcode, funct3, funct7, rs1, rs2, and rd fields; the Control Unit generates alu_sel, alu_src, and reg_write; the Register File provides source operands; the ALU computes the result; and the result is written back to the destination register and routed to the debug display.

Table 4.3 – Firmware instruction sequence and expected ALU results

Step	PC	Machine code	RISC-V instruction	Expected result
1	0x00	00F00093	addi x1, x0, 15	0x0000000F
2	0x04	00A00113	addi x2, x0, 10	0x0000000A
3	0x08	002081B3	add x3, x1, x2	0x00000019
6	0x14	40520333	sub x6, x4, x5	0x0000000F
9	0x20	0083F4B3	and x9, x7, x8	0x00000008
10	0x24	0083E533	or x10, x7, x8	0x0000000E
11	0x28	0083C5B3	xor x11, x7, x8	0x00000006

14	0x34	00D62733	slt x14, x12, x13	0x00000001
----	------	----------	-------------------	------------

- Firmware generation and loading. The firmware was written using a minimal C/inline assembly sequence. The generated firmware.hex file contains 32-bit RV32I machine instructions. In InstructionMemory.v, the statement \$readmemh("firmware.hex", rom) loads the machine code into a ROM array during simulation and synthesis. The address rom[pc[7:2]] is used because the PC is byte-addressed while the ROM is word-addressed.

-RTL Viewer verification. Quartus RTL Viewer confirms that TopCore_Firmware instantiates RiscV_ALU_Core and that the step_enable pulse generated from KEY[2] drives the execution of instructions. The visible core-level ports include debug_pc[31:0], debug_instr[31:0], debug_result[31:0], debug_rd[4:0], debug_alu_sel[2:0], and debug_reg_write.

- ModelSim verification. The firmware-based testbench tb_firmware_core.v executes the instruction sequence step by step. The transcript confirms that ADD, SUB, AND, OR, XOR, and SLT all return the expected results. The waveform also shows the relation between debug_pc, debug_instr, debug_alu_sel, and debug_result, which is used to compare the hardware result with the original firmware intent.

- Board-level verification. After setting TopCore_Firmware as the top-level entity and programming the DE2 board, KEY[3] resets the firmware core and KEY[2] steps one instruction at a time. SW[1:0] selects the HEX display mode: 00 shows debug_result, 01 shows debug_pc, 10 shows debug_instr, and 11 shows debug information including reg_write, rd, and alu_sel. In result mode, the HEX display must show 00000019 for ADD, 0000000F for SUB, 00000008 for AND, 0000000E for OR, 00000006 for XOR, and 00000001 for SLT.

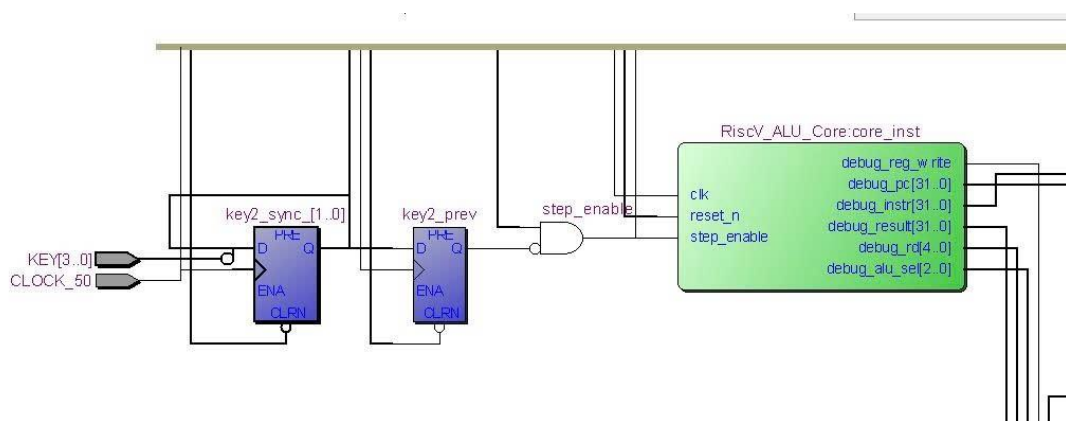


Figure 4.9 – RTL Viewer showing KEY[2] synchronization, step_enable, and RiscV_ALU_Core connection in TopCore_Firmware.

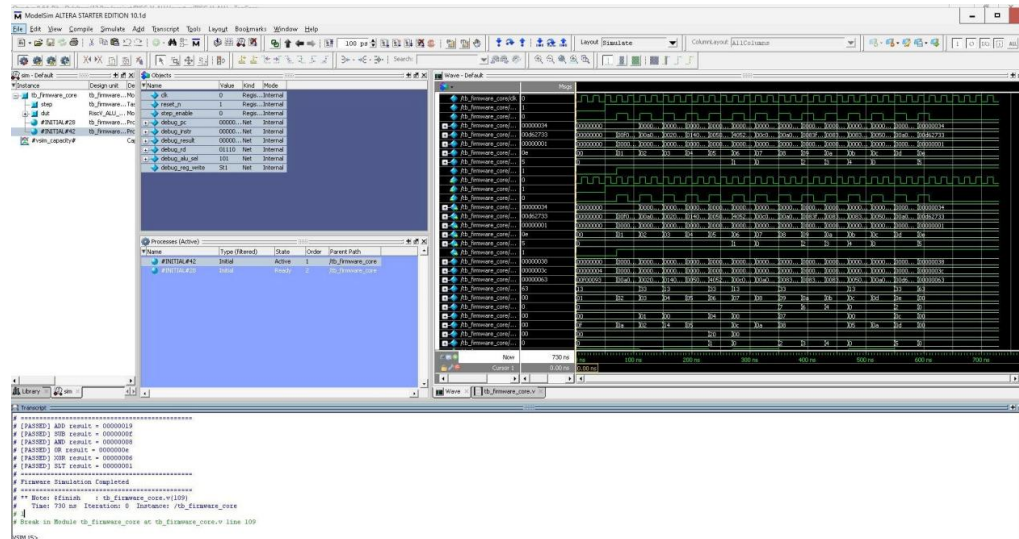


Figure 4.10 – ModelSim waveform and transcript for firmware-based RISC-V ALU core simulation.

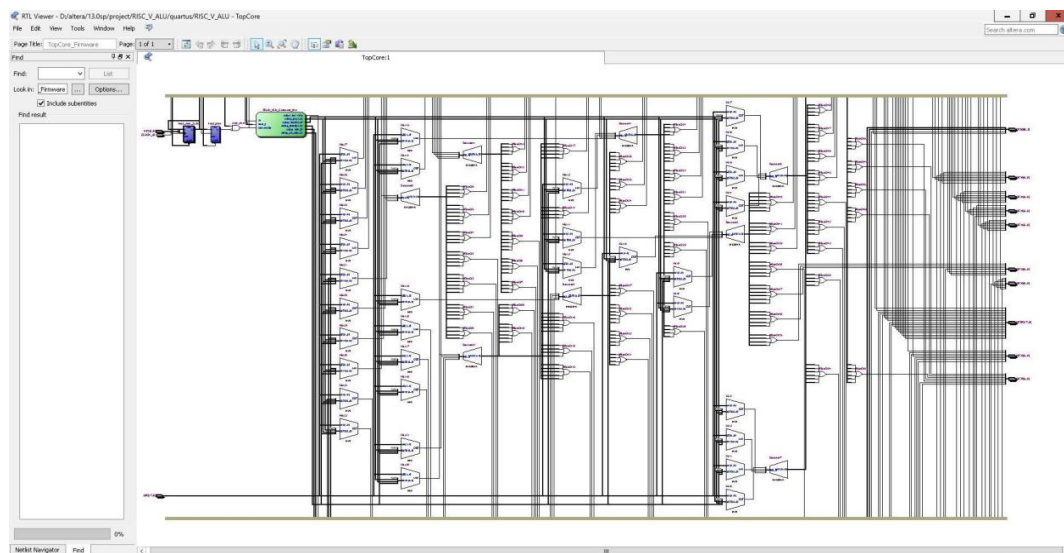


Figure 4.11 – Full RTL Viewer view of the synthesized firmware-based top-level design.

```

Transcript
# *****
# [PASSED] ADD result = 00000019
# [PASSED] SUB result = 0000000f
# [PASSED] AND result = 00000008
# [PASSED] OR result = 0000000e
# [PASSED] XOR result = 00000006
# [PASSED] SLT result = 00000001
# *****
# Firmware Simulation Completed
# *****
# ** Note: $finish      : tb_firmware_core.v(109)
#      Time: 730 ns  Iteration: 0  Instance: /tb_firmware_core
# 1
# Break in Module tb_firmware_core at tb_firmware_core.v line 109
VSI15>
    
```

Figure 4.12 – ModelSim transcript confirming PASSED status for ADD, SUB, AND, OR, XOR, and SLT.

CONCLUSION AND FUTURE DEVELOPMENT

Conclude

- This project has completed the design, simulation, firmware-based verification, and FPGA implementation of a 32-bit ALU system based on the RISC-V RV32I architecture on a DE2 FPGA board. Within the implementation scope, the ALU supports ADD, SUB, AND, OR, XOR, and signed SLT operations. The updated firmware-based flow demonstrates that machine instructions can be loaded from firmware.hex, fetched by a Program Counter, decoded by a Control Unit, executed through the Register File and ALU datapath, and displayed on the DE2 board through debug modes.
- Besides the computing core, the project provides two hardware demonstration modes. The manual mode uses SW[17:0] and KEY[3:0] to load operands and execute operations directly. The firmware mode uses TopCore_Firmware, where KEY[3] resets the core, KEY[2] steps through one instruction per press, and SW[1:0] selects result, PC, instruction, or debug display on HEX7-HEX0.
- However, the current system is still limited to a compact ALU-oriented RV32I subset. The firmware core supports the instruction sequence required to validate ADDI and the selected R-type ALU operations, but it does not yet implement a complete processor with data memory, branch control, load/store instructions, or full RV32I coverage.

Development direction

- In the future, the system can be expanded by adding the remaining RV32I ALU operations such as SLL, SRL, SRA, and SLTU; implementing a full instruction decoder with branch, load/store, and jump support; adding Data Memory and a complete write-back path; and replacing the step-by-step firmware execution mode with continuous clocked execution or a multi-cycle FSM for clearer instruction-stage observation.

REFERENCES

Books and textbooks

- [1] David A. Patterson & John L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 2nd Edition, Morgan Kaufmann, 2020.
- [2] Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd Edition, Pearson Education, 2003.
- [3] Stephen Brown & Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design, 2nd Edition, McGraw-Hill Education, 2008.
- [4] Pong P. Chu, FPGA Prototyping by Verilog Examples – Xilinx Spartan-3 Version, Wiley-IEEE Press, 2008.

Technical documentation

- [5] RISC-V International, The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20191213, 2019.
- [6] Intel (Altera), Cyclone II Device Handbook, Volume 1: Device Interfaces and Integration, Version 2.2, 2007.
- [7] Terasic Technologies, DE2 Development and Education Board User Manual, Version 1.7, 2008.
- [8] Intel Quartus Prime Pro Edition User Guide: Design Compilation, UG-20132, 2020.

Online documents

- [9] RISC-V Foundation, "RISC-V ISA Specifications," <https://riscv.org/technical/specifications/>, accessed June 2025.

[10] FPGA4Student, "RISC-V ALU Design in Verilog,"
<https://www.fpga4student.com>, accessed June 2025.